Tarea Corta 1

Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Bases de Datos II (IC 4302)
Primer Semestre 2026

---

1. **Objetivo General**

➔ Implementar una solución de observabilidad para bases de datos SQL y NoSQL.

---

2. **Objetivos Específicos**

➔ Instalar y configurar motores de bases de datos SQL y NoSQL mediante Kubernetes.

➔ Instalar y configurar una solución de monitoreo y alertas utilizando Prometheus y Grafana.

➔ Implementar pruebas de carga sobre bases de datos SQL y NoSQL mediante el uso de la herramienta Gatling.

➔ Instrumentar una aplicación para exponer métricas mediante Prometheus.

---

3. **Datos Generales**

➔ El valor de la tarea es: 10%

➔ La tarea debe ser implementada de forma individual o en grupos de máximo 5 personas.

➔ La fecha de entrega es viernes 27 de marzo del 2026 antes de las 11:59 pm.

➔ Cualquier indicio de copia será calificado con una nota de 0 y será procesado de acuerdo con el reglamento. La copia incluye código/configuraciones que se puede encontrar en Internet y que sea utilizado parcial o totalmente sin el debido reconocimiento al autor.

➔ La revisión es realizada de forma virtual mediante citas en las cuáles todos los y las integrantes del grupo deben estar presentes, el proyecto debe estar completamente automatizado con Helm, el profesor decide en que computadora probar.

➔ Se debe incluir documentación con instrucciones claras para ejecutar el proyecto.

➔ Se espera que todos y todas las integrantes del grupo entiendan la implementación suministrada.

➔ Se deben seguir buenas prácticas de programación en el caso de que apliquen. Por ejemplo, documentación interna y externa, estándares de código, diagramas de arquitectura, diagramas de flujo,

pruebas unitarias son algunas de las buenas prácticas que se esperan de un estudiante de Ingeniería en Computación.

➔ El lenguaje de implementación, en caso de ser requerido y no especificado por el profesor, deberá ser seleccionado por cada grupo y debe quedar debidamente documentado.

➔ Toda documentación debe ser implementada en Markdown.

➔ Si la instalación de servicios no se encuentra automatizado se obtendrá una nota de 0. Si el proyecto no se entrega completo, la porción que sea entregada debe estar completamente automatizada, de lo contrario se obtendrá una nota de 0.

➔ En caso de no entregar documentación se obtendrá una nota de 0.

➔ El email de entrega debe contener una copia del proyecto en formato tar.gz y un link al repositorio dónde se encuentra almacenado, debe seguir los lineamientos en el programa de curso.

➔ Los grupos de trabajo deben autogestionarse; la persona facilitadora del curso no es responsable si un miembro del equipo no realiza su trabajo.

➔ En consultas presenciales, por correo electrónico o por Telegram, como mínimo se debe haber leído sobre el tema y haber intentado comprenderlo. Las consultas deben ser concretas y se debe demostrar que se intentó resolver el problema en cuestión.

---

4. **Descripción**

Debido a la explosión en el uso de microservicios en los últimos años, al desarrollo de aplicaciones cada vez más interactivas y a la adopción de tecnologías como Internet of Things (IoT), se han comenzado a generar grandes cantidades de datos de monitoreo. Entre estos datos se encuentran métricas de sistemas, logs y traces. Esta situación limita la capacidad de que seres humanos y sistemas computacionales convencionales (como las bases de datos SQL) puedan extraer información valiosa que permita, entre otras cosas, entender y predecir el comportamiento de los sistemas, con el fin de tomar las medidas pertinentes a tiempo para asegurar su máxima disponibilidad y evitar actuar de forma reactiva.

Con la generación de grandes volúmenes de datos de series temporales, se han desarrollado bases de datos NoSQL especializadas llamadas Time Series Databases. Una de ellas es Prometheus, que ha evolucionado hasta convertirse en la solución open source líder para monitoreo. Esta herramienta se integra con otras soluciones open source como Kubernetes, Grafana y Thanos para formar una suite de aplicaciones capaz de competir con soluciones SaaS líderes en el mercado, como Datadog, New Relic y Dynatrace.

Para este trabajo, cada uno de los grupos deberá llevar a cabo las siguientes tareas con el fin de implementar una solución de monitoreo:

Instalación de motores de bases de datos

Para el desarrollo de esta tarea, cada uno de los grupos deberá elaborar Helm Charts que permitan la instalación y configuración de las siguientes herramientas:

➔ Prometheus

➔ Grafana

➔ MariaDB con un mínimo de 3 instancias (1 primary y 2 replicas).

➔ CouchDB

➔ MongoDB con 3 replicas.

➔ Elasticsearch

➔ Redis

➔ PostgreSQL

Se recomienda la elaboración de los siguientes Helm Charts:

➔ **bootstrap**: creará todos los namespaces y realizará la instalación de operadores.

➔ **databases**: contendrá las siguientes herramientas:

- MariaDB
- MongoDB
- Elasticsearch
- PostgreSQL
- CouchDB
- Redis

El uso del motor de base de datos deseado puede controlarse mediante dependencias y condiciones a nivel de Helm Charts. Por ejemplo, después de ejecutar `helm create databases`, se puede agregar el siguiente código al archivo Chart.yaml.

```yaml
dependencies:
  - name: kafka
    version: "26.8.5"
    repository: https://charts.bitnami.com/bitnami
    condition: kafka.enabled
  - name: mariadb
    version: "16.0.2"
    repository: https://charts.bitnami.com/bitnami
    condition: mariadb.enabled
```

Y luego agregar en el archivo values.yaml lo siguiente:

```yaml
elastic:
  enabled: true
  version: 8.6.1
  replicas: 1
  name: ic4302
kibana:
  enabled: true
  version: 8.6.1
  replicas: 1
  name: ic4302
kafka:
  enabled: true
mariadb:
  enabled: true
```

➔ **monitoring-stack**: contendrá las herramientas Prometheus, Grafana y Thanos, en caso de que se desee utilizar.

➔ **app**: contendrá los Helm Charts requeridos para la aplicación que se debe instrumentar para probar las bases de datos que no exponen una interfaz HTTP.

➔ **grafana-config**: cargará dinámicamente los dashboards y datasources.

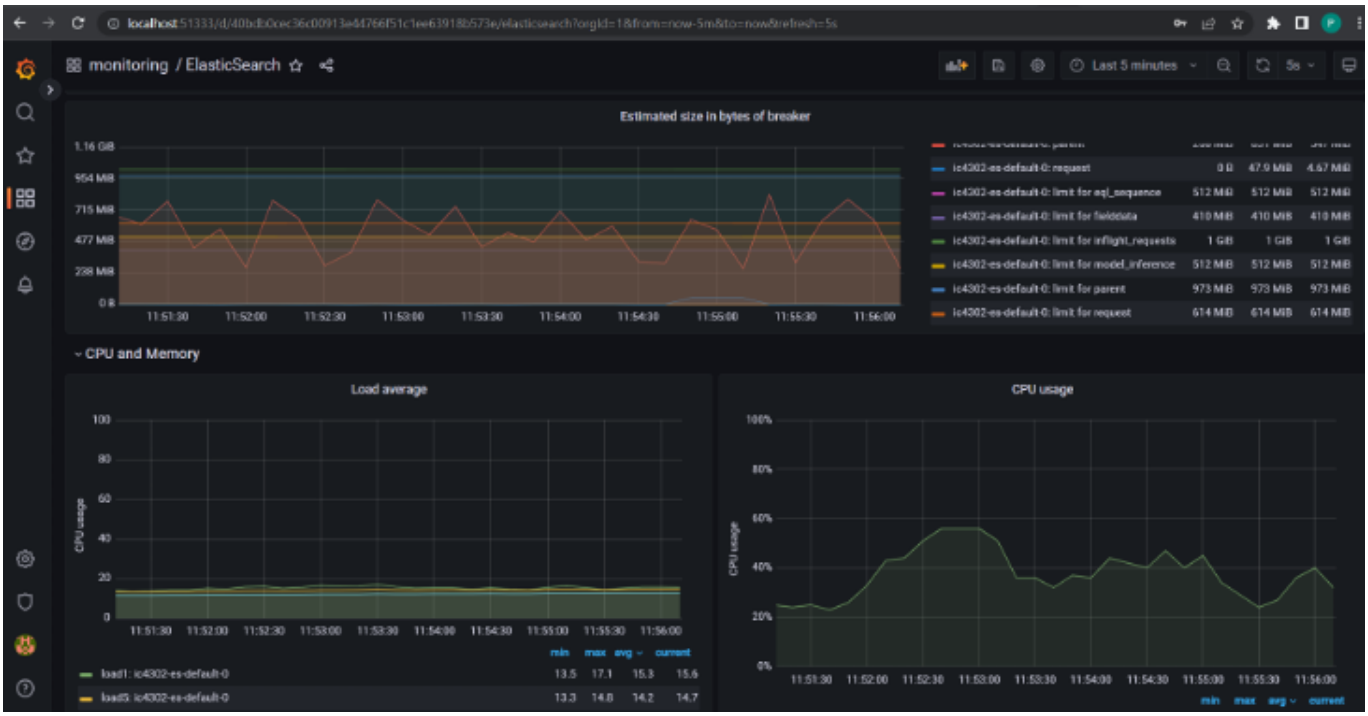
Es importante mencionar que cada uno de los paquetes de instalación proporcionados por Bitnami se encuentra preparado para generar métricas de uso, las cuales pueden ser utilizadas por Prometheus de forma dinámica (auto-discovery). Para habilitarlo, basta con utilizar los parámetros correctos en el Helm Chart.

Configuración de Grafana

Las configuraciones de Grafana se cargarán automáticamente mediante los Helm Charts que se instalarán. Cada grupo deberá identificar cuáles son los dashboards más adecuados para visualizar los datos; estos pueden encontrarse en https://grafana.com/grafana/dashboards/.

El profesor brindará un ejemplo de configuración que muestra cómo realizar esta tarea utilizando Elasticsearch.

El siguiente es un ejemplo de cómo se verá un dashboard:



Generación de pruebas

Cada uno de los grupos deberá generar pruebas de carga con Gatling. Estas se realizarán sobre los motores de bases de datos y deberán incluir los siguientes tipos de operaciones:

➔ Creación de registros/documentos.

➔ Borrado de registros/documentos.

➔ Actualización de registros/documentos.

➔ Búsquedas de registros/documentos.

Las pruebas deberán ejecutarse durante largos periodos de tiempo (al menos 30 minutos), con el fin de generar gráficos de monitoreo que permitan observar el comportamiento de los diferentes motores de bases de datos. Es importante observar, cuando sea posible, los siguientes parámetros (no todas las bases de datos los exponen):

➔ Disco.

➔ Memoria.

➔ CPU.

➔ Red (Network).

➔ IOPS.

➔ Open Connections.

➔ Queries per second.

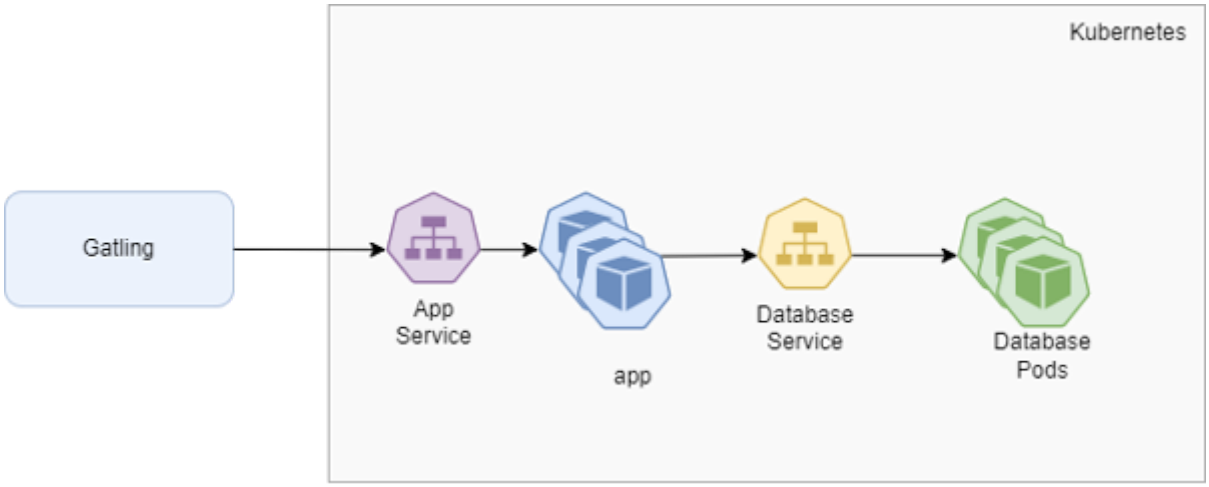
➔ Query response time.

➔ Thread Pools.

➔ File Descriptors.

La idea detrás de las pruebas de carga es medir el rendimiento de las bases de datos bajo una configuración específica. Se recomienda utilizar el mismo conjunto de datos en diferentes motores para poder realizar comparaciones. Es importante mencionar que Gatling se ejecutará fuera de Kubernetes.

Gatling es una herramienta que realiza pruebas de carga sobre endpoints HTTP. Sin embargo, no todas las bases de datos SQL y NoSQL exponen una interfaz para interactuar mediante este protocolo. Por esta razón, se deberá implementar una aplicación intermediaria, la cual se ilustra en la siguiente imagen.



La aplicación intermediaria deberá estar implementada en Python utilizando Flask, será administrada mediante un Deployment de Kubernetes y estará expuesta a través de un servicio de tipo ClusterIP.

Además, esta aplicación deberá estar instrumentada para exponer métricas a Prometheus. Se puede utilizar este ejemplo básico de instrumentación en el que únicamente se exponga el número de peticiones HTTP.

---

5. **Documentación**

La documentación debe incluir al menos:

➔ Guía de instalación y uso de la tarea: se debe explicar en detalle cómo ejecutarla y cómo utilizarla.

➔ Configuración de las herramientas: haciendo énfasis en los valores utilizados para cada una.

➔ Pruebas de carga realizadas: se debe especificar el tipo de datos que se están almacenando (dataset), el tipo de prueba (creación, borrado, actualización y búsqueda), los parámetros utilizados (configuración de Gatling), los resultados (apoyados por la información de monitoreo y los gráficos obtenidos) y las conclusiones de cada prueba. Se deben incluir al menos 5 pruebas por motor de base de datos.

➔ Conclusiones y recomendaciones de la tarea corta.

---

6. **Recomendaciones**

➔ Utilizar telnet/curl/postman para interactuar con los componentes y verificar que los mismos están funcionando correctamente.

➔ Utilizar Docker Desktop para instalar Kubernetes en Windows.

➔ Utilizar Minikube para ejecutar Kubernetes en Linux

➔ Realicen peer-review/checkpoints semanales y no esperen hasta el último momento para integrar su aplicación.

➔ Crear un repositorio de GitHub e invitar al profesor.

➔ Realizar commits y push regulares.

---

7. **Entregables**

➔ Documentación.

➔ Proyecto de Gatling con todos los scripts requeridos..

➔ Aplicación intermediaria.

➔ Helm Charts para la instalación de las herramientas requeridas por la tarea.

---

8. **Evaluación**

Rubro	**Valor**
Helm Charts	10%
Pruebas Gatling	45%
Aplicación intermediaria	15%
Documentación	30%